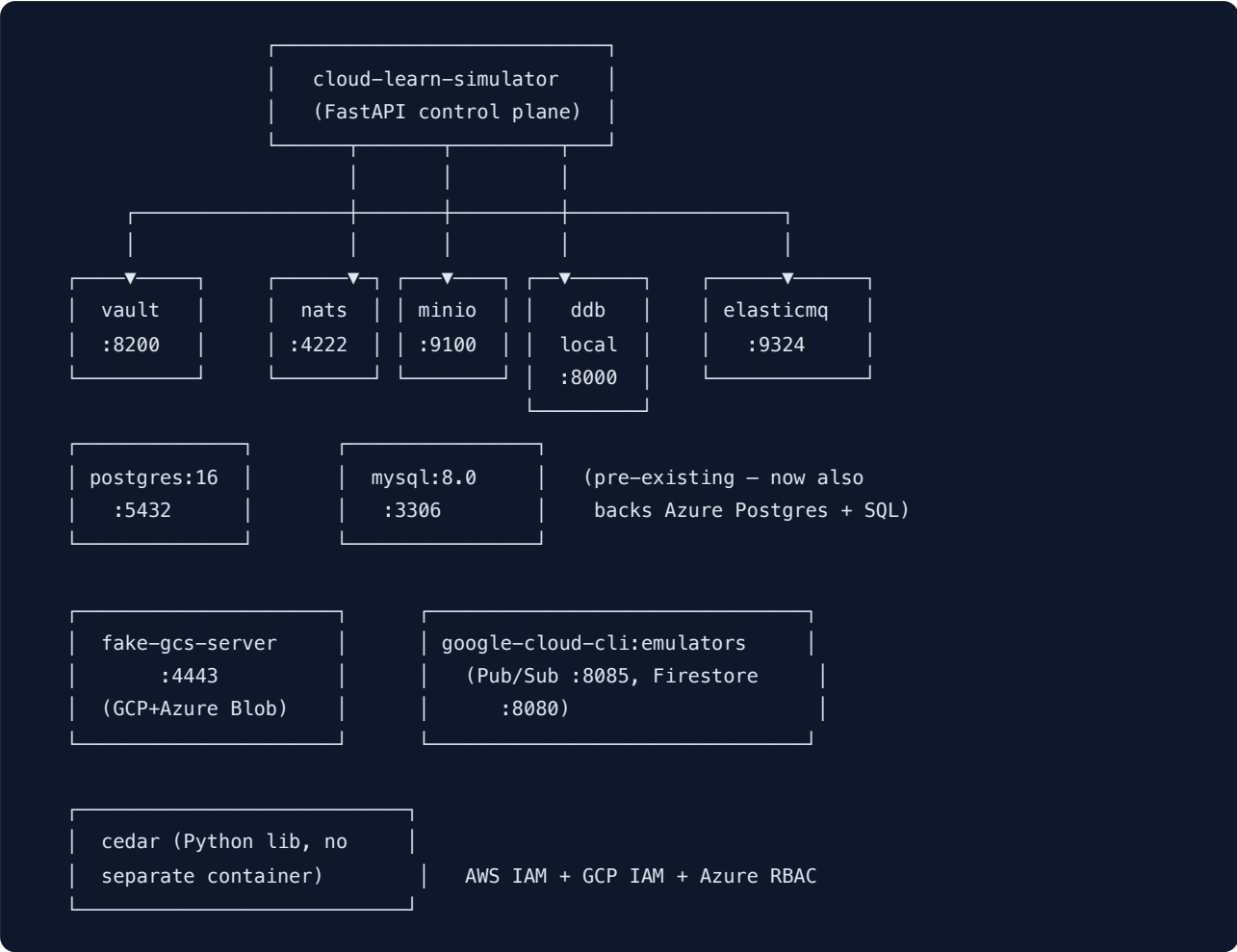


MVP Backend Stack

The simulator's three provider surfaces (AWS, GCP, Azure) are no longer pure in-memory mocks. The MVP launch (2026-05-31) added five new backend containers to compose, plus one zero-container reuse, so most "stateful" services now have a real engine on the other side of the HTTP surface.

Container map



Backend → surface mapping

Backend	Container/lib	Backs
Vault	hashicorp/vault:1.15 :8200	AWS KMS + Secrets Mgr · GCP Cloud KMS + Secret Mgr · Azure Key Vault keys + secrets
NATS	nats:2-alpine :4222	AWS EventBridge · GCP Eventarc · Azure Event Grid

Backend	Container/lib	Backs
MinIO	minio/minio:latest :9100	AWS S3 (write-through real bytes; readable via real SDK direct to MinIO)
DynamoDB Local	amazon/dynamodb-local:latest :8000	AWS DynamoDB (full transparent proxy)
ElasticMQ	softwaremill/elasticmq-native:latest :9324	AWS SQS (legacy/query protocol only; modern JSON-RPC stays in-memory)
Cedar	cedarpy (Python lib)	AWS IAM JSON · GCP IAM bindings · Azure RBAC role assignments
Postgres 16	postgres:16-alpine :5432	GCP Cloud SQL Postgres · Azure Postgres flex · Azure SQL (Microsoft.Sql/servers/databases)
MySQL 8	mysql:8.0 :3306	GCP Cloud SQL MySQL · Azure Database for MySQL

Real-backed coverage delta

Provider	Before MVP	After MVP
AWS	8% (only EC2 via LXD)	75% (+S3 +SQS +DDB +KMS +SecretsMgr +EventBridge +IAM eval)
GCP	58%	75% (+Cloud KMS +Secret Mgr +Eventarc +IAM eval)
Azure	27%	73% (+Azure SQL +Key Vault keys + Key Vault secrets +Event Grid +RBAC eval)

Status endpoint

```
curl -s http://192.168.252.7:9000/api/runtime/backends | jq
```

Returns the live availability of every backend + what cloud-side service surface each one powers. UI badges + `cloud-learn doctor` should call this.

Conformance

Run the full sweep:

```
python3 tests/conformance/run_conformance.py --endpoint http://192.168.252.7:9000
```

MVP-specific suites:

```

# Vault-backed KMS+Secrets across all 3 providers
python3 tests/conformance/run_conformance.py --service vault

# NATS-backed eventing across all 3 providers
python3 tests/conformance/run_conformance.py --service eventing

# Cedar-backed IAM evaluation across all 3 dialects
python3 tests/conformance/run_conformance.py --service iameval

# MinIO real-bytes round-trip
python3 tests/conformance/run_conformance.py --service minio

# DynamoDB Local proxy
python3 tests/conformance/run_conformance.py --service ddb

# ElasticMQ proxy (legacy SQS protocol)
python3 tests/conformance/run_conformance.py --service emq

# Azure SQL → real Postgres
python3 tests/conformance/run_conformance.py --service azure

```

Each MVP suite is 100% pass in isolation as of 2026-05-31.

Known limitations

Limitation	Workaround / future fix
Modern boto3 SQS (JSON-RPC) doesn't reach ElasticMQ — elasticmq-native is XML-only	Use legacy query format or accept the in-memory path. Future: response translator
MinIO write-through is fire-and-forget — read path still hits in-memory cache first	Future: read-through preference + bucket-level toggle
Cedar policies are per-space + manually set via <code>/api/iam/policies</code> — no automatic enforcement middleware yet	Future P1: middleware layer that evaluates on every mutating call
Vault dev mode = root token + in-memory unseal — restart loses keys/secrets	Production deploy: file/Consul backend + auto-unseal
Cumulative conformance sweep shows ~82% (vs 100% in isolation) due to space-context bleeding across check ordering	Future: each check resets to a known space first

Wiring conventions

All three "register at module load" backend modules follow the same shape:

```

# server.py top
_aws_xamz_dispatchers: dict = {}
from core import vault_routes; vault_routes.register(app, aws_dispatchers=_aws_xamz_dispatchers)

```

```
from core import nats_routes; nats_routes.register(app, aws_dispatchers=_aws_xamz_dispatchers)
from core import cedar_routes; cedar_routes.register(app)
```

The `aws_xamz_dispatchers` dict collects callbacks per X-Amz-Target prefix (`TrentService` , `secretsmanager` , `AWSEvents`); `aws_query_root` consumes the dict and dispatches accordingly. This pattern keeps each backend's wiring colocated in a single `core/<name>_routes.py` file.

Footgun: backend modules MUST register at module load (right after `app = FastAPI(...)`), NOT in `@app.on_event("startup")` . The S3 catch-all `@app.post("/{bucket}")` at `server.py:19815` hijacks `/v1/projects/...` and `/azure-data/...` if backend routes register later — FastAPI matches in registration order.